Interview Brief — CubeTwin / iTelematics Satellite Engineering Internship

For: Soham Vinchu · TY B.Tech ENTC, KIT's College of Engineering, Kolhapur **Interview:** 29 September 2026 (tomorrow) · likely remote **Role:** Satellite Engineering Internship — *CubeTwin: CubeSat Digital Twin Simulation* **Source:** AICTE listing · CubeTwin platform

> *Read section 1 and section 9 tonight. Everything else is depth you can dip into.*

1. The 60-second version (know this cold)

Who they are. iTelematics Software Private Limited — a small Bangalore R&D company (incorporated 2012, 2–10 people) whose commercial work is EV telematics and battery intelligence: battery State-of-Health, capacity fade, internal-resistance rise, thermal-runaway detection, SOH/RUL prediction models, telematics data pipelines. CubeTwin sits under their **EV Society™ / EV.ENGINEER™** education-and-research wing, inside the **ISHAVASYAM.ORG** space-station research initiative.

What the internship actually is. You join the team building **CubeTwin** — a browser-based educational digital-twin simulator for CubeSat *energy, mission and reliability*. You'd follow their published 12-week student R&D roadmap (14 Sep – 6 Dec 2026): orbital-period function → power budget → battery MVP → voltage/temperature → mission modes & scheduling → fault injection → telemetry & reliability metrics → 3D view → Monte Carlo uncertainty → validation workbook → production polish → release.

The technical core. Orbit period → sunlight/eclipse split → solar generation through PMAD → load schedule → battery SOC integration → terminal voltage and cell temperature → safe-mode hysteresis → fault injection → reliability metrics under uncertainty.

The terms. Remote / virtual, 3 months, 30 seats, pan-India, **unpaid**, 13 academic credits, certificate on successful completion. Batch 1 = 1 Oct–30 Dec 2026; Batch 2 = 1 Jan–30 Mar 2027. Listing says start 31 Dec 2026 (inconsistent with Batch 1 — ask). Interview stage = "a short conversation **or a take-home task**".

Why you fit. You are an ENTC student who builds energy-monitoring embedded systems (INA219 current/voltage sensing, load cells, ESP8266 telemetry, OLED/Blynk monitoring) *and* who models a digital system before building it (SV-16: RTL + Simulink model + FPGA target + PCB). You also build web front-ends (React/Vite/Tailwind/Firebase), which is exactly what a *browser-based* simulator platform needs. Their stack is your stack plus orbit mechanics.

2. What they will actually be listening for

Their listing names 15 desired attributes: communication (verbal & written), entrepreneurial, presentation, listening, management, creative, time management, teamwork, learning ability, research-oriented, startup experience, hands-on experience, out-of-box thinking, problem solving, coding.

But the *web platform they built* tells you the real bar more precisely. Everything on their page screams these five values:

Their value	Evidence on their page	How you win the point
Verification discipline	Weekly "Verify" tasks: unit tests, dimensional checks, hand-calc vs code within a stated tolerance, "no divide-by-zero, NaN or infinite output", deterministic output for same scenario + seed	Say: "I hand-calculate first, then assert the code matches within a tolerance."
Radical honesty about limits	"Educational R&D prototype · Simulated data · Not flight software", "Simulation results are not flight evidence", "no fabricated ratings, partners or awards", "do not assign a TRL without criteria and evidence"	Never oversell. Volunteer what your model *cannot* do before they ask.
Traceability	Assumptions register, requirement-to-test traceability table, version number + changelog, references with revision dates	Show you document *why*, not just *what*.
Accessibility + rigour in the UI	Screen-reader chart summaries, reduced-motion mode, 2D fallback when WebGL is unavailable, print stylesheet, 360/390/412/768 px layouts	You have real web experience — this is rare in electronics applicants.
Learn → Calculate → Build → Verify → Deliver	Their whole 198-task structure	Answer questions in that order, out loud.

The single best thing you can do in this interview: prove you actually read and understood their roadmap, not just the job description. Two or three specifics from the platform (e.g. the truth/observed SOC split, the hysteresis inequality `safe entry < reserve ≤ recovery ≤ maximum SOC`, the energy-accounting residual) will separate you from every other applicant.

---

3. Your positioning: the honest, strong version

Your differentiator in one line:

> *"I'm an electronics student who builds hardware and writes the model of it first — my Simulink/RTL model of the processor I'm building is effectively a digital twin of my own hardware. CubeTwin is the same discipline applied to a spacecraft power system, which is where I want to work."*

Why you, three concrete reasons:

- **Energy/power sensing is already your thing.** Your saline monitoring system measures current, voltage and power with an INA219, weighs load cells through an HX711, and pushes telemetry to a display, Blynk and a Telegram bot. That is a miniature EPS-with-telemetry: generation/consumption sensing, local display, remote monitoring, alerts. CubeTwin's whole subject is energy accounting and telemetry.
- **You model before you build.** SV-16: you wrote an original 16-bit ISA, the RTL (≈80% implemented and tested), a Simulink datapath model, and you hold a fixed FPGA target and a PCB pipeline. You understand that a model is only useful if it is verified against something independent.
- **You can ship the front end too.** React + Vite + Tailwind + Firebase means you can contribute to the browser simulator — charts, CSV/JSON export, state handling — not just the maths. Most ENTC applicants cannot.

Your honest gaps (say them before they ask — it reads as maturity):

- No aerospace coursework / no CubeSat-specific experience. *(Fix: you have read their model and can quote its equations and defaults.)*
- Your modelling is in **MATLAB/Simulink**, not Python/JS. *(Fix: you have a plan — week 1 is one pure function plus unit tests; you'll have Python foundations in place.)*

- No prior STK / Orekit / GMAT / Valispace exposure. *(Fix: you know they are listed as "future references for higher-fidelity comparisons" on their own page.)*
- No formal requirements-traceability practice. *(Fix: you keep an assumptions register for SV-16 — board-stack choices, component availability, cost — and can describe it.)*

---

4. Tonight's plan (about 3 hours, highest yield first)

Block A — 45 min. Memorise the numbers in section 5 and the eight formulas. Write them by hand once. Actually compute two of them on paper (500 km period, 35-min eclipse energy). Hand-writing is what makes them survive under pressure.

Block B — 45 min. Skim their platform properly. Open aerospace.ishavasyam.org/space/cubesat. Run the baseline simulation. Then:

- Change altitude 500 → 600 km, watch the period change (~94.5 → ~96.5 min).
- Run **Solar degradation** and note the min-SOC change.
- Run **Heater stuck on** and find the first safe-mode event.
- Set **SOC sensor bias** to +15 pp and compare true vs observed SOC.
- Run Monte Carlo twice with the same seed and confirm identical output.

Write three observations you can say out loud. That's your "I already used your platform" answer.

Block C — 45 min. Rehearse out loud, standing, once:

- The 60-second intro (section 6).
- The SV-16 pitch in 2 minutes, ending on the *digital-twin* analogy (section 7).
- "Why CubeTwin?" (section 6).
- Your weakest question — "Do you know Python?" — out loud, three times, until it stops feeling awkward.

Block D — 45 min. Read the 30 questions in section 8. Don't memorise answers word-for-word; memorise the *shape* of each answer. Highlight the six you'd dread, and practise only those.

Morning (30 min): read the one-page cheat sheet (`04-cheatsheet.md`). Do not learn anything new after that. Test camera, mic, lighting, internet backup.

---

5. The numbers you must know cold

Reference mission (their published defaults)

Parameter	Value
Form factor	3U CubeSat, LEO
Altitude	500 km
Orbital period	94.5 min (exact: 94.47 min)
Sunlight / eclipse per orbit	≈59.5 min / ≈35 min
Inclination	51.6° (descriptive only in their "duration mode")
Peak solar / delivered to bus	20 W / 18 W (90% PMAD)
Battery energy capacity	40 Wh
Initial SOC	80%
Reserve / safe-mode entry	30% / 20%

Parameter	Value
Nominal / essential-only load	8 W / 3 W
Payload activity	starts at 4 h · 18 W · 10 min
Downlink activity	starts at 5 h · 14 W · 8 min
Simulation	24 h at 10 s step (max 20,000 steps per run)
Thermal bus / initial temp	20 °C / 20 °C

Worked values you should be able to produce live

Question	Working	Answer
Period at 500 km	$T = 2\pi\sqrt{(a^3/\mu)}$, a = 6371+500	**94.47 min**
Period at 600 km	a = 6971 km	**96.54 min** (+2.07)
Eclipse at 500 km, β = 0	$\arccos(\sqrt{(h^2+2Rh)}/((R+h)\cos\beta))/\pi$	**37.8% ≈ 35.7 min**
Energy in eclipse, 8 W × 35 min	8 × 35/60	**4.67 Wh** (4.91 Wh drawn at 95%)
SOC after 1 h at 8 W	32 − 8/0.95 = 23.58 Wh of 40	**58.95%**
Voltage sag	1 A × 0.12 Ω	**0.12 V**
Delivered solar	20 W × 0.90	**18 W**
Equivalent full cycles (their baseline)	152.64 Wh throughput ÷ (2 × 40 Wh)	**1.908 EFC**

The eight equations (all from their lesson cards)

```
T  = 2*pi*sqrt((Re + h)^3 / mu)                      Re = 6371 km, mu = 398600.4418 km^3/s^2
Psolar = G * A * eta_cell * derating * max(0, cos θ)   G = 1361 W/m^2 at 1 AU
Pnet   = Psolar_delivered - Pload
E_new  = E + eta_c * Pnet * dt/3600            (charging)
E_new  = E + Pnet * dt / (3600 * eta_d)        (discharging, Pnet < 0)
SOC    = 100 * E / usable_capacity ;  DoD = 100 - SOC
Vterm  = OCV(SOC) - I * Rinternal
dT     = dt/C_th * (I^2 * R + Q_ext - (T - T_bus)/R_th)
controller:  safe_entry < reserve <= recovery <= maximum SOC
```

Battery-sizing recipe (be ready to derive it from scratch)

- Energy the battery must supply during eclipse = `P_eclipse × t_eclipse`.
- Divide by discharge efficiency → energy actually drawn from the pack.
- Divide by the **allowed depth of discharge** (the design choice: 20%, 30% — lower DoD = longer cycle life, bigger battery) → **minimum usable capacity**.
- Never plan to use the last of the pack: keep a reserve (they use 30%) that only safe mode may touch.
- Cross-check the recharge side: over one orbit, energy generated in sunlight must exceed (loads in sunlight + energy drawn in eclipse + losses). If not, the battery walks down orbit by orbit — that is exactly what the **per-orbit energy margin** column in their tool shows.

6. Ready-to-speak answers

60-second intro

> *"I'm Soham Vinchu, a third-year Electronics and Telecommunication student at KIT's College of Engineering, Kolhapur. I came in through a diploma in ENTC with 92.47%, so I've had a lot of hands-on lab time.*

> *My work splits into two halves that I'm now trying to join up. On the hardware side I build energy-monitoring embedded systems — my most recent one measures saline load cells and the system's own current and voltage with an INA219, then streams telemetry to an OLED, a Blynk dashboard and a Telegram bot. On the modelling side I'm building **SV-16**, an original 16-bit processor architecture: I wrote the ISA, about 80% of the RTL is implemented and tested, I have a Simulink model of the datapath, and the target is a Lattice ECP5 FPGA with a PCB in progress. So I write the model of the system first, then build the artefact and verify it against the model.*

> *CubeTwin is the same discipline applied to a spacecraft power system — energy generation, storage, loads and reliability — and it's browser-based, which is a bonus because I also build web front-ends with React and Tailwind. That's why this specific internship is the one I want."*

Why CubeTwin? (the answer that lands)

> *"Three reasons. One: the subject is energy — solar generation, battery state of charge, eclipse energy balance — and that's the part of electronics I keep gravitating to, because it's measurable and it's unforgiving. Two: your platform is explicitly honest about what it is — an educational prototype, not flight software, with verification separate from validation. I'd rather learn engineering under that discipline than in a place that overclaims. Three: I read your 12-week roadmap, and week 1 is 'write one pure function that calculates orbital period, and unit-test it against a hand calculation'. That is exactly how I like to work — calculate by hand first, then assert the code reproduces it."*

What will you contribute?

> *"Immediately: verification work. I hand-calculate, then test the code against the calculation — orbital period, constant-power battery cases, energy-balance closure — and I write the assumptions register. In parallel I can help on the front end, since I've built React/Vite/Tailwind interfaces with charts and Firebase before, and your roadmap has CSV/JSON export, accessibility summaries and print stylesheet work. And I'm comfortable with the hardware layer, so if a fault is describing a physical mechanism — internal resistance rise, I^2R heating, capacity fade — I can sanity-check that the model's numbers are physically plausible rather than just self-consistent."*

If they ask something you don't know

> *"I don't know that off the top of my head. Here's how I'd find out: [name the source or the derivation you'd start from]. May I reason it out loud with you?"*

Never bluff orbital mechanics or orbit-determination jargon. They will spot it instantly, and their whole culture is anti-overclaiming. "I'd start from X and check it against Y" scores better than a confident wrong answer.

---

7. Your project pitches (mapped to their domain)

SV-16 — your flagship (2 minutes)

- **The problem I set myself:** design an original 16-bit processor — instruction set, datapath, control — and take it all the way to a physical board, not just a simulation.
- **What exists:** ISA and instruction encoding defined; ALU operations (ADD/SUB, plus AND/OR/XOR/NOT/shifts/MUL/DIV/INC/DEC in the plan); register file and status system; memory interface; instruction fetch; roughly 80% of RTL implemented and tested; Simulink model for

architecture-level work.

- **Design philosophy:** reliability over compactness, simplicity over features, buildability over theoretical optimisation, easily obtainable parts, no BGA, hand-assemblable packages.
- **Where it's going:** synthesis and bitstream for the Lattice ECP5 (LFE5U-12F-6TG144C), a PCB, programming/configuration memory, then processor-level tests and hardware validation.
- **The CubeTwin link (say it explicitly):** "*SV-16 taught me that a model of a system is only trustworthy when you can check it against something independent — a hand calculation, a unit test, or the physical board. That's the same reason I like that your platform separates physical truth from observed telemetry from detector status: the model, the measurement, and the conclusion are three different things, and mixing them up is how you fool yourself.*"

Saline monitoring system → their EPS/telemetry world (60 seconds)

- **Sensing:** 3 load cells via HX711 for the fluid column; INA219 for the system's own bus current, voltage and power.
- **Compute + display:** ESP8266, OLED plus I²C LCD, buzzer alerts, servo control.
- **Telemetry:** Blynk IoT dashboard + Telegram bot notifications.
- **Cost engineering:** built around ₹1,700, with a ₹1,350 variant once the INA219 route was reconsidered.
- **The link:** "*Every part of that is a tiny version of a spacecraft power problem: measure what you generate and consume, decide what the state is, display it, alert when it goes out of bounds, and keep the whole thing inside a cost and power budget.*"

MSP430 battery node → power-path thinking (30 seconds)

MSP430F5529 LaunchPad + ESP8266 + INA219 + OLED + DC motor + rechargeable battery pack. Real embedded debugging: Energia toolchain, MCU programming, hardware interfacing, a battery-powered supply rail. Analog foundations (op-amps, ADC/DAC, feedback, gain) transfer directly to sensing chains, OCV measurement and signal conditioning.

Web work (30 seconds)

React + Vite + Tailwind + Firebase, deployed sites, real debugging (white screens, build/deploy failures, dependency issues, localhost/proxy problems). "*A simulator platform is half physics engine and half interface. I can work on both halves, which I think matters for a project whose roadmap includes chart accessibility, CSV export and a printable workbook.*"

---

8. Thirty questions they may ask (with the shape of a good answer)

A. Motivation & background

- **Tell me about yourself.** → Section 6 intro, 60 seconds, then stop talking.
- **Why this internship, and why space?** → Section 6 "Why CubeTwin".
- **You're an ENTC student — why should we take you for a CubeSat energy project?** → "Because the electrical power system *is* electronics: solar conversion, DC/DC regulation, battery chemistry, sensing, telemetry, load management. My daily coursework is exactly that; what's new to me is orbital geometry, which is one formula and a shadow fraction."
- **What do you know about CubeTwin?** → 3U LEO educational digital-twin simulator; energy/mission/reliability; truth vs observed vs detector; fault experiments; Monte Carlo; not flight software, not calibrated.
- **Which of our 12 weeks interests you most, and why?** → Pick week 6 (fault injection & diagnosis) or week 10 (validation & workbook) and justify — both are verification-heavy.
- **Are you comfortable with a 3-month remote commitment alongside your degree?** → Give a concrete weekly number and how you'll protect it (see section 11).

B. Fundamentals (high probability)

- **What is a CubeSat, and what is a "U"?** → Standardised small satellite; 1U ≈ 10 cm cube, mass limit ~1.33 kg per the Cal Poly CDS; 3U ≈ 10×10×34 cm, ≤4 kg; larger formats (6U/12U) exist and dispenser ICDs govern the details. Standardisation is why students and small teams can reach orbit at all.
- **What subsystems make up a CubeSat bus?** → EPS, OBC, C&DH, ADCS, TT&C (comms), thermal control, structure; the payload performs the mission.
- **What is the EPS and what is PMAD?** → The subsystem that generates, stores, conditions and distributes electrical power; PMAD is the power management and distribution hardware that converts and routes it (e.g. array voltage to a regulated bus), including protection and switching.
- **Why is eclipse the sizing driver?** → No generation, but loads continue; the battery must bridge the shadow and still be rechargeable within the sunlight period. Worst case is $\beta = 0$ (orbit plane containing the Sun vector) — ~37.8% of a 500 km orbit in their simplified geometry.
- **Calculate the orbital period at 500 km.** → $T = 2\pi\sqrt{(a^3/\mu)}$ with $a = 6871$ km → 94.47 min. Say the formula before the number.
- **What happens if you move to 600 km?** → Higher altitude → longer period → 96.54 min. Also: longer eclipse in absolute minutes for the same fraction, and less drag.
- **How much energy does the spacecraft need during a 35-minute eclipse at 8 W?** → 4.67 Wh, and 4.91 Wh actually drawn from the pack at 95% discharge efficiency.
- **What is SOC and DoD?** → SOC = stored energy as a percentage of usable capacity; DoD = the percentage removed (100 − SOC in their simplified definition). Cycle life depends heavily on how deep you cycle.
- **Charge vs discharge efficiency — why do they matter?** → You store less than you supply and must draw more than the load receives; ignoring them makes a mission look ~5–10% better than it is, which is exactly the kind of optimism that kills small satellites.
- **What is OCV, and why can't you read SOC directly from voltage?** → Open-circuit voltage is measured at rest; under load it sags by I·R, and it flattens across the mid-SOC plateau, so voltage alone is a weak SOC indicator. Their model uses an editable **illustrative lookup** — explicitly a diagnostic approximation, not an observer.
- **Why does internal resistance rise matter?** → It increases sag (so you hit undervoltage sooner), increases I^2R heating, and reduces the usable energy — a classic ageing mechanism, which is literally the subject of your parent company's battery-SOH work.
- **What is C-rate, and what is EFC?** → C-rate normalises current to capacity (1C empties the pack in about an hour). Equivalent Full Cycle combines partial cycles into the energy equivalent of one full charge–discharge: their baseline 152.64 Wh through a 40 Wh pack → 1.908 EFC.
- **Explain the energy balance over one orbit.** → Sunlight: array powers loads *and* recharges. Eclipse: battery carries the loads. Over a full orbit, generation must exceed consumption plus losses, or SOC walks down. Their per-orbit margin table is exactly that check.
- **What is safe mode?** → A predefined low-risk state protecting essential functions after an anomaly or low energy; here the controller drops non-critical payload/downlink below the reserve and enters safe mode below the entry threshold.
- **Why hysteresis in the controller?** → Different entry and exit thresholds plus a dwell time stop rapid mode toggling; without it a system hovering near a threshold chatters between states. Their inequality: `safe entry < reserve ≤ recovery ≤ maximum SOC`. (You studied ON/OFF controllers and PID in Control Systems — say so; hysteresis is the same idea.)

C. Modelling, verification & reliability

- **What is a digital twin, and is CubeTwin one?** → A digital representation connected to the state and history of a *particular* physical system. CubeTwin is a physics-based simulation platform today; it becomes a *calibrated* twin only when linked to traceable data from a specific test article or spacecraft. They say so themselves — agreeing precisely is the strongest possible answer.

- **Verification vs validation?** → Verification: "did we build the model right?" (code implements the stated equations — unit tests, hand calculations, tolerance). Validation: "did we build the right model?" (do the equations represent the intended physical system — calibration against measured data). Their page: verification checks code against equations, validation checks equations against reality.
- **How would you test this simulator?** → (a) hand-calculate and compare; (b) close the energy balance and assert the residual is ~0; (c) unit tests on pure functions with explicit units; (d) edge cases — zero load, zero generation, empty battery, division by zero; (e) step-size independence; (f) determinism: same scenario + seed → identical output; (g) Monte Carlo collapse to the deterministic result when uncertainty is zero.
- **How do you know your numbers are right?** → I know they're *consistent*: the residual closes to ~5e-11 Wh, generated and consumed match the published run, and the analytical values match hand calculations. Whether they're *right* needs validation data — there is none here, and the platform says so.
- **A battery is at 3% SOC and the heater is stuck on. What happens?** → Loads exceed generation, the battery empties, you get unmet demand and undervoltage (brownout intervals), the controller should already be in safe mode, and mission activities fail. Unserved demand must be *recorded*, never invented — the model must not create energy.
- **What is FDIR?** → Fault Detection, Isolation and Recovery: identify an abnormal condition, locate the likely cause, and perform or advise a bounded response. Fault injection is not proof of detection.
- **Truth vs observed vs detected — why separate them?** → Truth is the authoritative simulated state; observed is what telemetry shows (biased, dropped out, quantised); detected is what a threshold-based logic concludes. A +15 pp SOC bias makes a spacecraft look healthy while it is actually below reserve. Their bias detector uses truth as a *teaching oracle* — real detection needs independent measurements.
- **What does the Monte Carlo probability mean?** → The fraction of seeded simulated trials that complete all activities with no unmet load or undervoltage, under the stated uniform input uncertainties. It is a *simulated* probability, not a prediction of real spacecraft reliability, and the confidence interval describes sampling uncertainty. Same seed → same distribution; zero uncertainty → deterministic result.
- **How would you improve the model?** → Strongest honest answer: (a) replace the "sunlight-then-eclipse" simplification with a real β-angle shadow geometry, then validate against an independent tool (they list GMAT and Orekit as future references); (b) replace the OCV lookup plus I·R approximation with a proper cell model and an SOC observer (coulomb counting with an EKF) so SOC is *estimated* rather than read out; (c) add temperature-dependent capacity and efficiency; (d) correlate against real telemetry from a test article — which is the step that turns a simulation into a calibrated twin.

D. Working style

- **How do you handle a bug you can't find?** → Bisect: reduce to the smallest failing case, check units and dimensions first, verify the physics by hand, then instrument. Mention a real one — the SV-16 Simulink register-block issues (MUXes, bit slicing, unit delay, data types, signal widths) that you solved by getting the register system working before adding control logic.
- **How do you document your work?** → Their pattern exactly: results, evidence, assumptions, and one next improvement — the "Deliver" line of every one of their weeks. Say you'd keep an assumptions register and a validation log.
- **Have you worked in a team / used Git?** → Yes: SV-16 has a second member owning the PCB; you use Git/GitHub; Technovation 2K25 Head and class representative show you can coordinate people. Add: "I'll state a small limitation honestly — I've mostly worked on my own repositories, so I'd want a code-review cadence so my commits are easy for others to review."

---

9. If they set a take-home task

The listing explicitly says the interview can be "*a short conversation or a take-home task*". Be ready for:

- "Write a function that calculates the orbital period and test it."
- "Model the battery SOC over one orbit and plot it."
- "Propose six power-fault scenarios and say which metric detects each."
- "Write the verification cases you'd use to prove this model."

You already have a working answer: `code/cubetwin_model.py` — a stdlib-only implementation of their published equations (orbit, eclipse, solar/PMAD, battery integration, voltage, thermal, mode hysteresis, fault injection, telemetry, Monte Carlo) plus 60 unit tests in `code/test_cubetwin_model.py`. Run it, read it, and be able to explain every line — if you show it, you must be able to defend it. Its verification results:

Check	Published	This model
Orbital period at 500 km	94.5 min (94.47 shown)	**94.47 min**
Period at 600 km	—	**96.54 min**
Energy generated, baseline 24 h	274.51 Wh	**274.50 Wh**
Energy consumed, baseline	194.52 Wh	**194.47 Wh**
Minimum SOC	80.0%	**80.07%** (both are the *initial* SOC — the pack never dips below its start)
Energy throughput	152.64 Wh	**150.98 Wh**
Equivalent full cycles	1.908	**1.887**
Energy-accounting residual	4.82e-11 Wh	**5e-11 Wh** (closes)
Charge/discharge loss	7.839 Wh	**8.21 Wh** (inferred charge eff. 0.97)
Determinism (same seed)	required	**verified**
Monte Carlo, ±10%, seed 42, 100 trials	—	**100% success, 95% Wilson CI 96.3–100%**

Be honest about the one place you don't match them: your final SOC is 96.95% versus their 91.81%, because their charge-efficiency default and rejected-charge accounting aren't published (you inferred 0.97 charge efficiency from their loss figure). *Saying that yourself, before they find it, is worth more than the match itself.*

Do not present this as "I built CubeTwin". Present it as: "*I reimplemented your published equations from your lesson cards to make sure I understood them, then tested my implementation against your published metrics. Here's where I match and here's where I had to infer.*"

---

10. Questions you should ask (pick 4–6)

Ranked by how much they signal that you read the platform:

- "Your 12-week workbook runs 14 September – 6 December, but the listing's start date is 31 December. How do interns joining Batch 1 fit against a roadmap that's already mid-flight, and which batch would you want me in?"
- "Week 3 mentions an 'ISAST-ready PDF' with a concept submission milestone around 30 September — what does that submission go to, and is it still open?"
- "Is each intern's deliverable a self-contained module against the scenario schema 1.0, or do we all work in one shared codebase with reviews?"
- "What does the code-review and demo cadence look like week to week? Is there a fixed weekly review with a mentor?"

- "Is the platform's engine written in JavaScript/TypeScript, or is there a Python side? I'd like to know what to be fluent in before I start."
- "How does the CubeSat work feed into the Space Station digital twin that ISHAVASYAM.ORG is developing — will interns ever touch the station-level model?"
- "For the certificate and the 13 credits, what evidence do you need from me — the weekly 'record your results, evidence, assumptions and one next improvement' entries, or specific deliverables?"
- "What would make an intern stand out enough for you to keep them on for a later project?"
- "You describe the platform as an educational prototype that isn't calibrated against a test article. Is there a plan to get real telemetry or a hardware-in-the-loop bench in future — and would interns be involved?"

Do not open with stipend, certificates or hours. Those are legitimate, but ask them at the end or by email afterwards.

---

11. Commitments, time and the honest risk questions

You have three live commitments at once: TY B.Tech coursework (EN and control/DSP/maths), the SV-16 milestone (your own target was 24 October 2026 for bitstream, PCB, assembly and validation), and GATE 2027 preparation. Plus this is **unpaid**.

Prepare this answer, because it will be asked in some form:

> *"I can commit about 10–12 hours a week, structured: two weekday evenings and a longer weekend block, plus whatever live review call you schedule. My SV-16 board work is partly delegated and my academic load is predictable this semester, so I can protect that. What I won't do is sign up for more than I can deliver, because on a 3-month programme with milestones, an intern who under-delivers is worse than one who was honest about capacity."*

Due diligence to do before accepting (not necessarily in the interview):

- Confirm what "13 credits" means at your college, and whether the college must approve the internship before you start.
- Ask what the certificate requires in practice (they state: active participation, reviews/workshops, progress documentation, deliverables, a demonstration and a final submission).
- Ask for one example of what a previous intern produced. If the answer is vague, weigh the certificate accordingly.
- If placement/internship cell approval is needed, start that paperwork tomorrow — batch 1 starts 1 October.
- Understand the trade you are making: unpaid, early-stage, small team — the return is a real project in a real repo, a space-domain specialisation, mentor review, and academic credit. It is *not* a brand-name résumé line. If you want that, ask whether top performers get credited or continued work.

---

12. Logistics checklist (do this tonight and in the morning)

- [ ] Video call: camera at eye level, light in front of you, plain background, wired or charged headset.
- [ ] Backup internet: phone hotspot tested, and tell them in advance if you may need to switch.
- [ ] Quiet room, phone on silent, family told not to knock.
- [ ] Keep open: your CV, this brief, `04-cheatsheet.md`, the CubeTwin page in a tab, your GitHub.
- [ ] Notebook and pen ready for any live calculation.
- [ ] Screen-share rehearsed: terminal running `python3 cubetwin_model.py`, plus SV-16 repo.

- [ ] Water, a charger, and 10 minutes of quiet before the call.
- [ ] Join 2 minutes early. Test your mic *in the call UI*.
- [ ] Have your questions (section 10) written down — asking two good questions changes the tone of the whole interview.

Prepared from the AICTE listing, the CubeTwin platform documentation (aerospace.ishavasyam.org/space/cubesat, last reviewed 12 September 2026), ishavasyam.org, itelematics.com and your profile in `me`*. Verify any figure before you quote it as fact, and never claim a skill you cannot demonstrate on the spot.*